Breadth-First Search (BFS)

AIM: To write a Python program to implement Breadth-First Search for traversing a graph level by level.

Algorithm:

1. Start the program.
2. Initialize the graph, starting node, and an empty visited set.
3. Insert the starting node into a queue and mark it as visited.
4. Remove the front node from the queue.
5. Display or process the removed node.
6. Add all unvisited adjacent nodes to the queue and mark them as visited.
7. Repeat Steps 4–6 until the queue becomes empty or the target node is found.
8. Display the traversal order.
9. Stop the program.

Code :

exp07_bfs.py > ...

```python
from collections import deque

def bfs(graph, start_node):
    visited = set()
    queue = deque([start_node])
    visited.add(start_node)
    traversal_order = []

    while queue:
        node = queue.popleft()
        traversal_order.append(node)

        for neighbor in graph.get(node, []):
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

    return traversal_order

def bfs_shortest_path(graph, start, goal):
    visited = {start}
    queue = deque([[start]])

    while queue:
        path = queue.popleft()
        node = path[-1]

        if node == goal:
            return path

        for neighbor in graph.get(node, []):
            if neighbor not in visited:
                visited.add(neighbor)
                new_path = list(path)
                new_path.append(neighbor)
                queue.append(new_path)
```

Output :

```
PS D:\CSA1702-AI\Experiments> & C:\python311\python.exe d:/CSA1702-AI/Experiments/exp07_bfs.py
BFS Traversal starting from A:
A -> B -> C -> D -> E -> F

Shortest path from A to F:
A -> C -> F
PS D:\CSA1702-AI\Experiments>
```

Result :

Thus, the Python program to implement Breadth-First Search (BFS) was executed successfully.